

Partially Observable RL with Memory Traces

Onno Eberhard¹² · Michael Muehlebach¹ · Claire Vernade²

¹Max Planck Institute for Intelligent Systems ²University of Tübingen



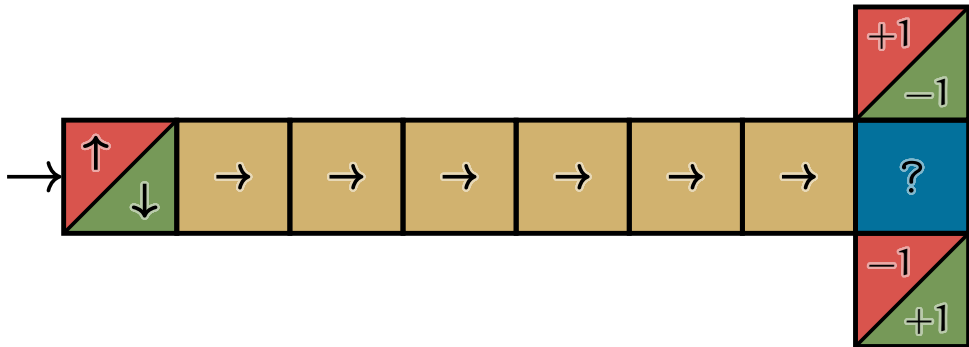
EBERHARD KARLS
UNIVERSITÄT
TÜBINGEN



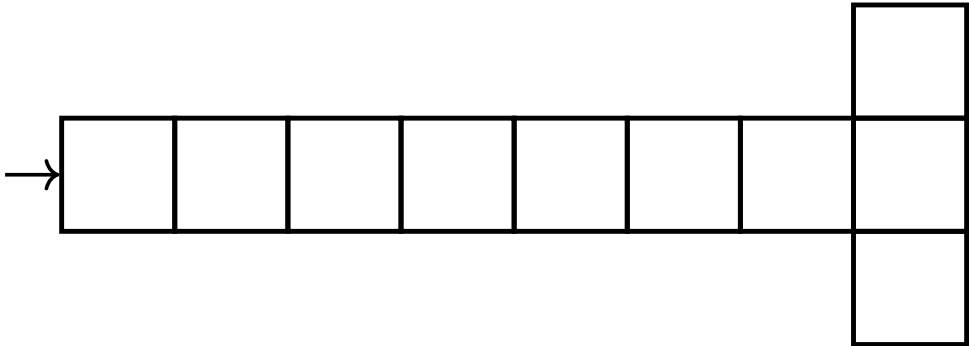
imprs-is

9 Oct 2026 · Edmonton, Canada

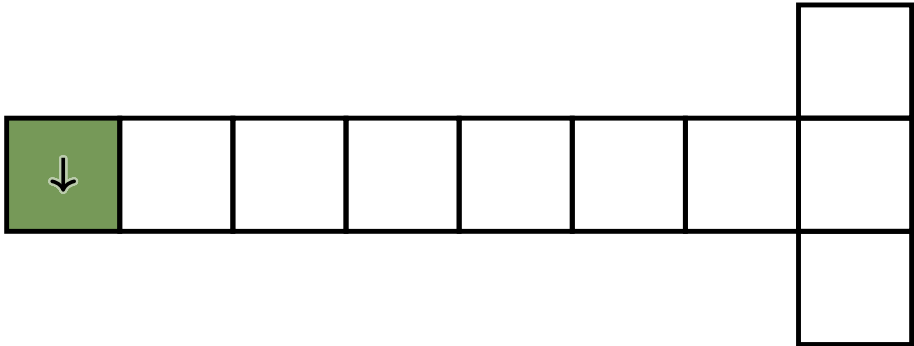
Bakker's T-maze



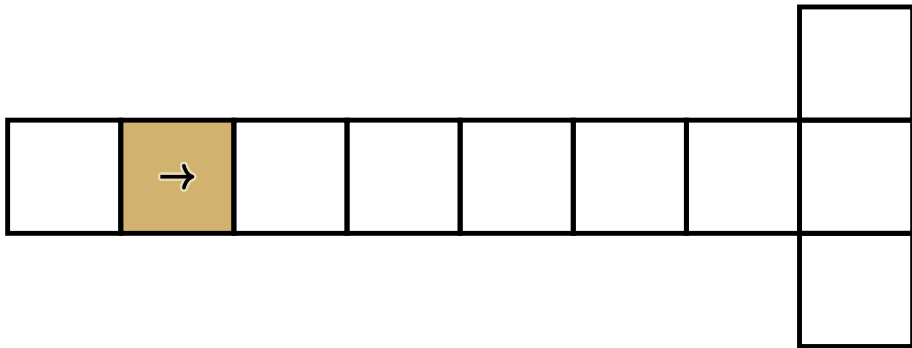
Bakker's T-maze



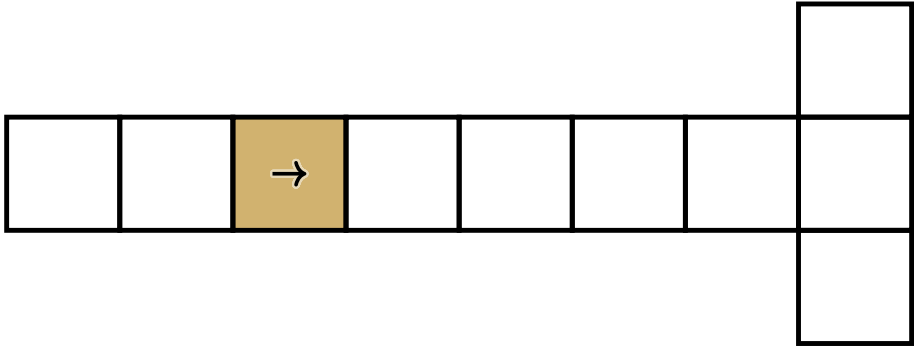
Bakker's T-maze



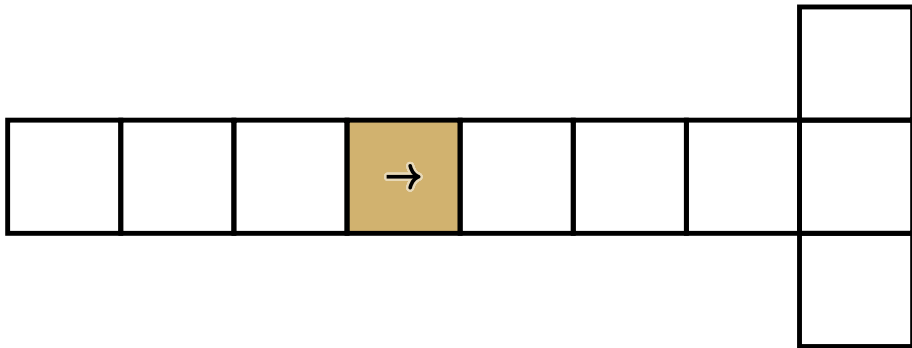
Bakker's T-maze



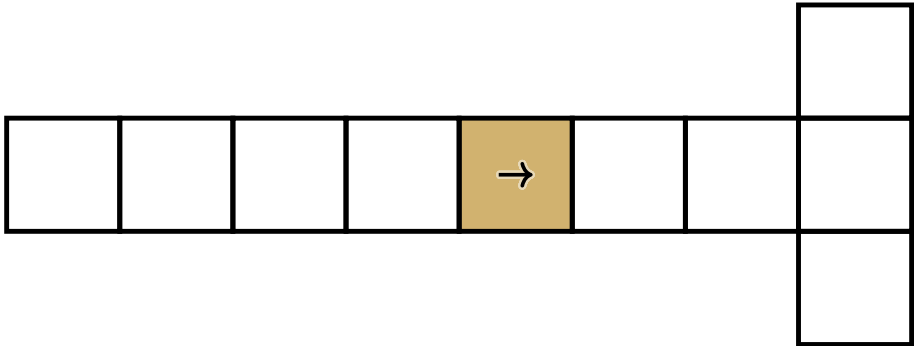
Bakker's T-maze



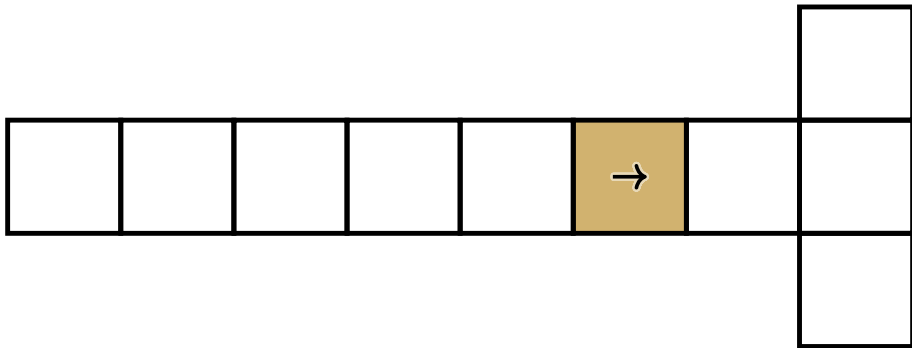
Bakker's T-maze



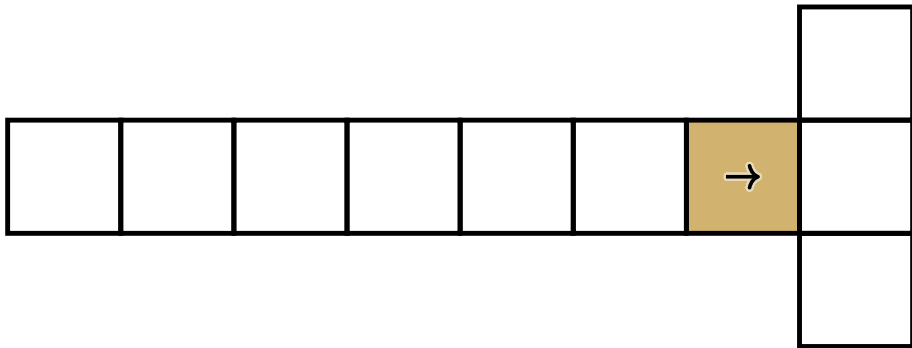
Bakker's T-maze



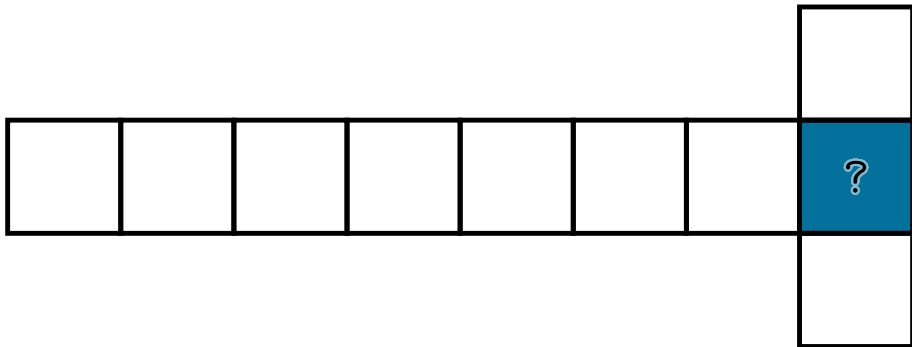
Bakker's T-maze



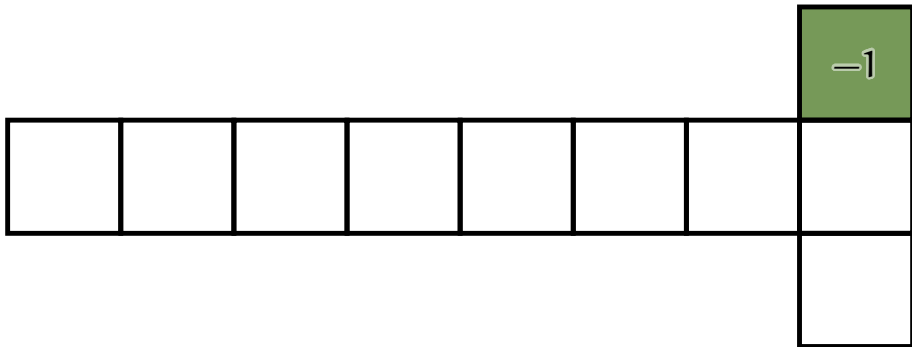
Bakker's T-maze



Bakker's T-maze



Bakker's T-maze



Memory

- ▶ Memory is necessary in many partially observable environments
 - *Memory*: a compressed representation of the history of observations

Memory

- ▶ Memory is necessary in many partially observable environments
 - *Memory*: a compressed representation of the history of observations
- ▶ Length- m window (“frame stacking”):

$$\text{win}_m(\underbrace{y_0, y_{-1}, \dots}_{\text{history } h}) \doteq (y_0, y_{-1}, \dots, y_{-m+1})$$

Memory

- ▶ Memory is necessary in many partially observable environments
 - *Memory*: a compressed representation of the history of observations
- ▶ Length- m window (“frame stacking”):

$$\text{win}_m(\underbrace{y_0, y_{-1}, \dots}_{\text{history } h}) \doteq (y_0, y_{-1}, \dots, y_{-m+1})$$

- ▶ Our approach: the *memory trace* with forgetting factor $\lambda \in [0, 1)$:

$$z_\lambda(h) \doteq (1 - \lambda) \sum_{k=0}^{|h|-1} \lambda^k y_{-k}$$

Memory

- ▶ Memory is necessary in many partially observable environments
 - *Memory*: a compressed representation of the history of observations
- ▶ Length- m window (“frame stacking”):

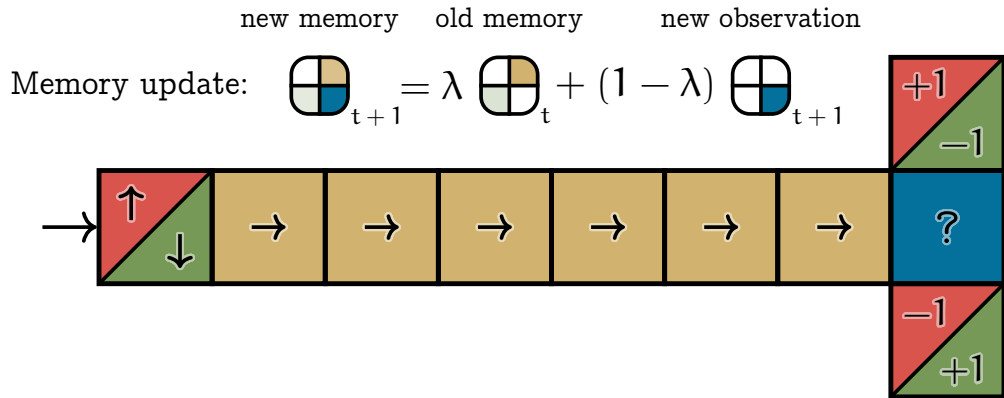
$$\text{win}_m(\underbrace{y_0, y_{-1}, \dots}_{\text{history } h}) \doteq (y_0, y_{-1}, \dots, y_{-m+1})$$

- ▶ Our approach: the *memory trace* with forgetting factor $\lambda \in [0, 1]$:

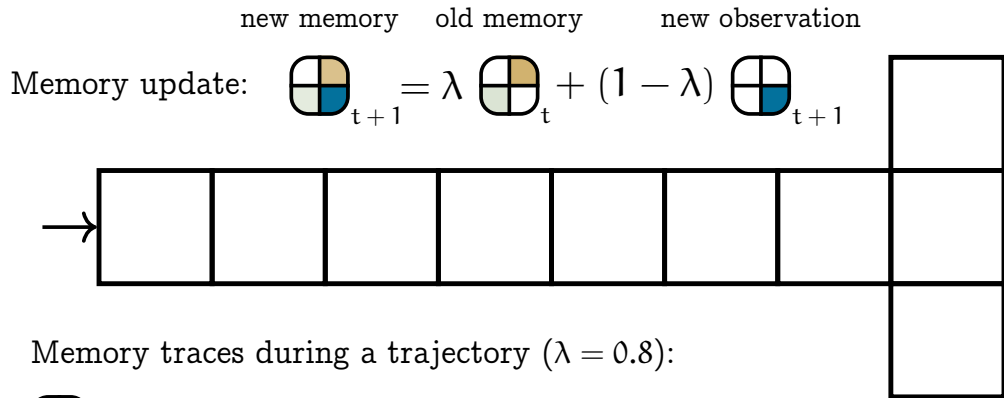
$$z_\lambda(h) \doteq (1 - \lambda) \sum_{k=0}^{|h|-1} \lambda^k y_{-k}$$

→ Recursively computable: $z_\lambda(y_0, y_{-1}, \dots) = \lambda z_\lambda(y_{-1}, y_{-2}, \dots) + (1 - \lambda)y_0$

Memory traces in the T-maze



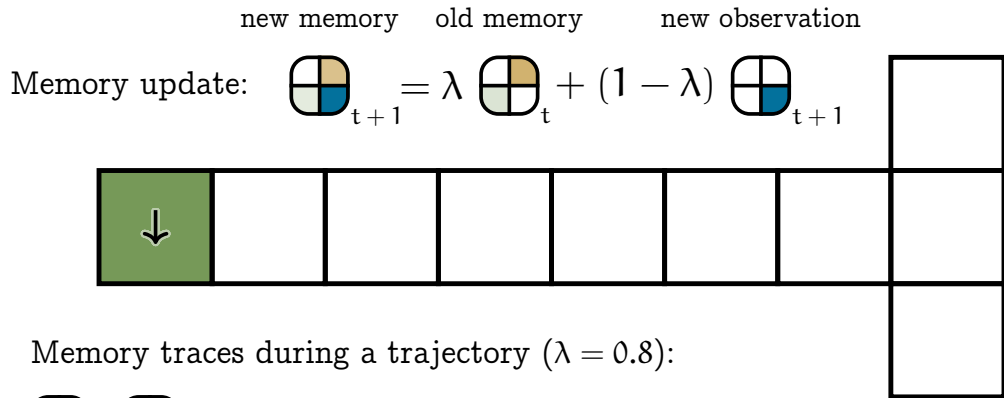
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):

$$\bigoplus_0$$

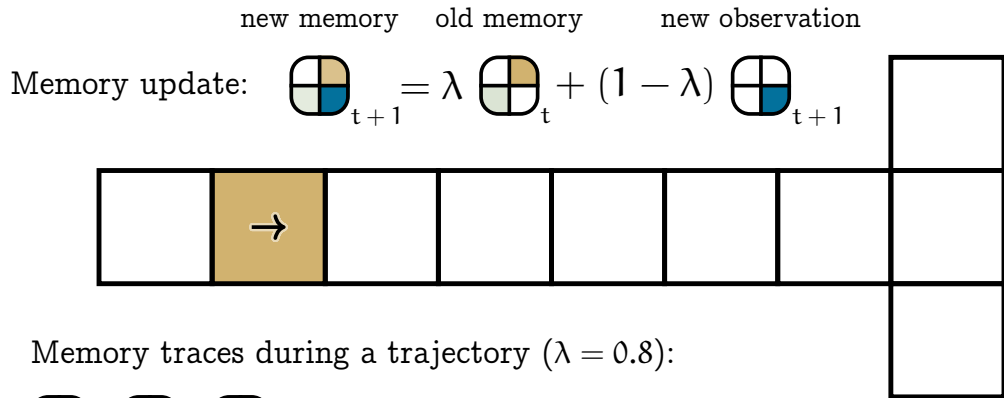
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):

$$\bigoplus_0 \quad \bigoplus_1$$

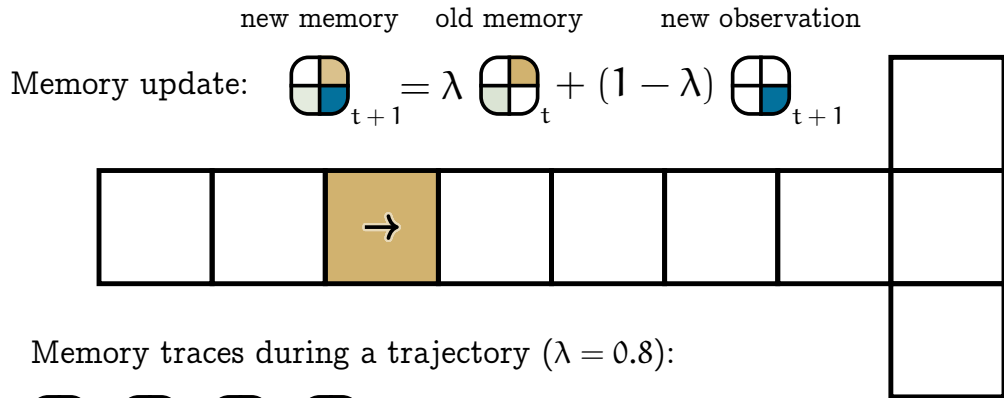
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



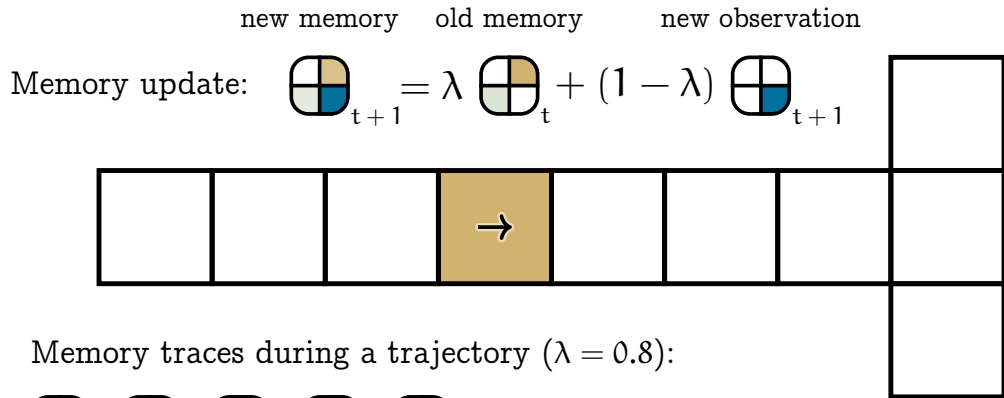
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



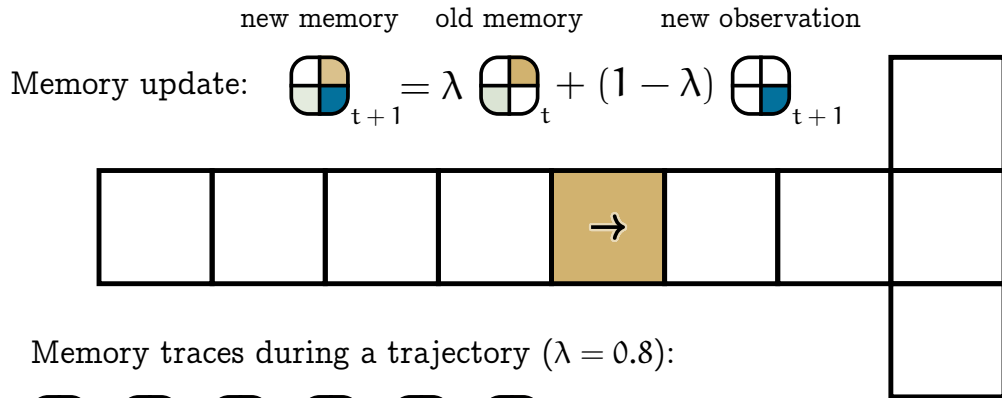
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



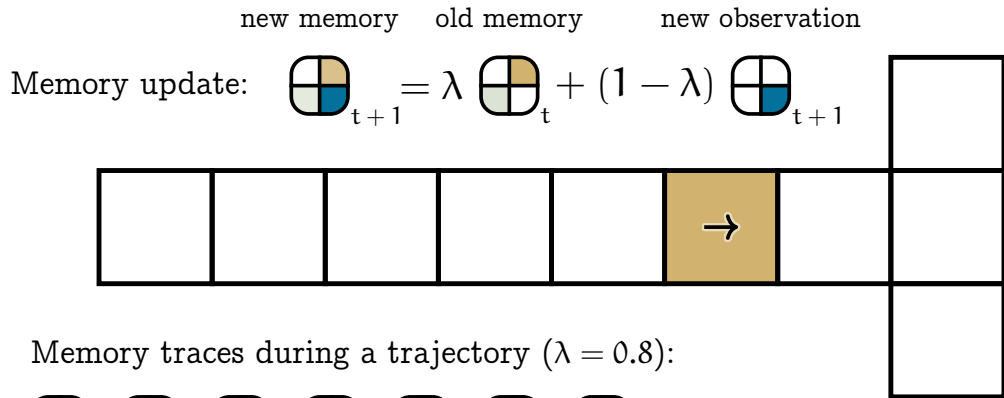
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



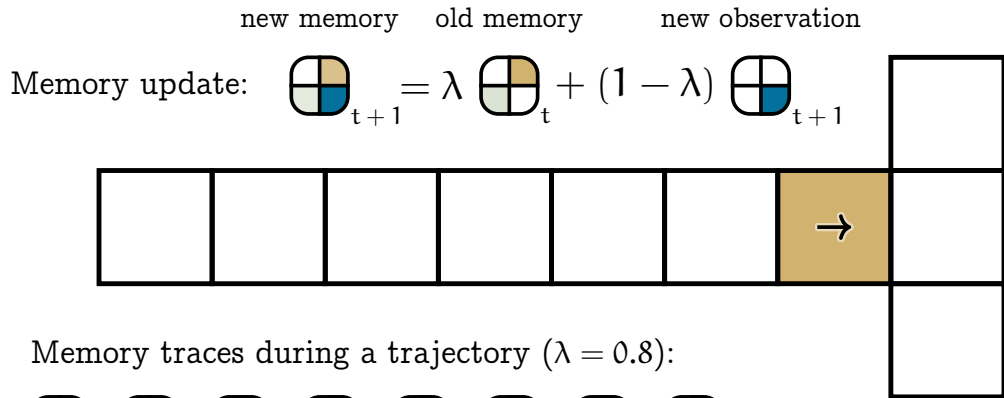
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



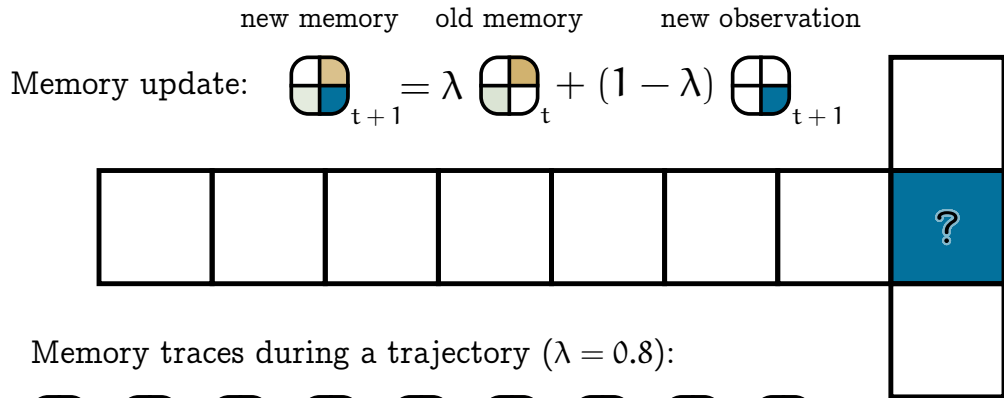
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



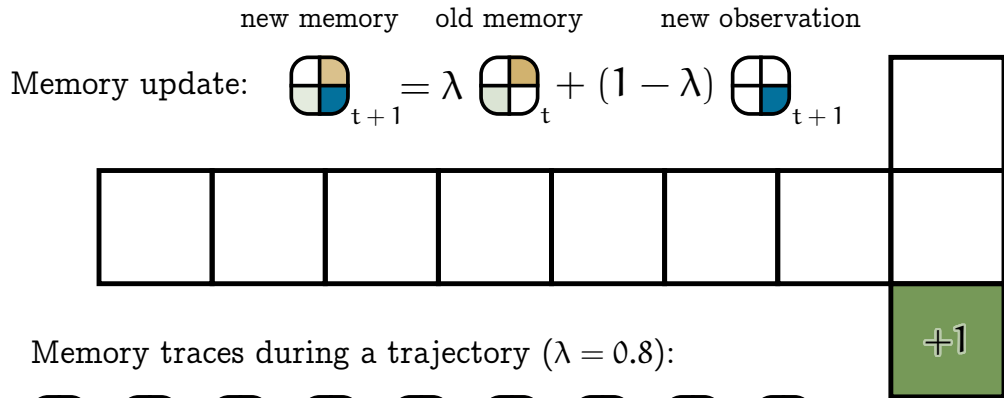
Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



Memory traces in the T-maze



Memory traces during a trajectory ($\lambda = 0.8$):



Offline on-policy evaluation

- We consider the problem of *policy evaluation* with offline data
 - The environment $\mathcal{E}$ is a hidden Markov model (no explicit policy)
 - The observation space $\mathcal{Y}$ is one-hot

Offline on-policy evaluation

- ▶ We consider the problem of *policy evaluation* with offline data
 - The environment $\mathcal{E}$ is a hidden Markov model (no explicit policy)
 - The observation space $\mathcal{Y}$ is one-hot
- ▶ Q: How much data do we need to accurately estimate the value function?

Offline on-policy evaluation

- ▶ We consider the problem of *policy evaluation* with offline data
 - The environment $\mathcal{E}$ is a hidden Markov model (no explicit policy)
 - The observation space $\mathcal{Y}$ is one-hot
- ▶ Q: How much data do we need to accurately estimate the value function?
- ▶ Given a function class $\mathcal{F} \subset \{\mathcal{Y}^\infty \rightarrow [\underline{v}, \bar{v}]\}$, find $f \in \mathcal{F}$ that minimizes

$$\mathcal{R}_{\mathcal{E}}(f) \doteq \mathbb{E}_{\mathcal{E}} \left[\left\{ f(y_0, y_{-1}, \dots) - \sum_{t=0}^{\infty} \gamma^t r(y_{t+1}) \right\}^2 \right],$$

where $r : \mathcal{Y} \rightarrow [\underline{r}, \bar{r}]$ (rewards), $\gamma \in [0, 1)$ (discount), $\underline{v} \doteq \underline{r}/(1 - \gamma)$, and $\bar{v} \doteq \bar{r}/(1 - \gamma)$

Offline on-policy evaluation

- ▶ We consider the problem of *policy evaluation* with offline data
 - The environment $\mathcal{E}$ is a hidden Markov model (no explicit policy)
 - The observation space $\mathcal{Y}$ is one-hot
- ▶ Q: How much data do we need to accurately estimate the value function?
- ▶ Given a function class $\mathcal{F} \subset \{\mathcal{Y}^\infty \rightarrow [\underline{v}, \bar{v}]\}$, find $f \in \mathcal{F}$ that minimizes

$$\mathcal{R}_{\mathcal{E}}(f) \doteq \mathbb{E}_{\mathcal{E}} \left[\left\{ f(y_0, y_{-1}, \dots) - \sum_{t=0}^{\infty} \gamma^t r(y_{t+1}) \right\}^2 \right],$$

where $r : \mathcal{Y} \rightarrow [\underline{r}, \bar{r}]$ (rewards), $\gamma \in [0, 1)$ (discount), $\underline{v} \doteq \underline{r}/(1 - \gamma)$, and $\bar{v} \doteq \bar{r}/(1 - \gamma)$

- ▶ Window memory: $\mathcal{F}_m \doteq \{f \circ \text{win}_m \mid f : \mathcal{Y}^m \rightarrow [\underline{v}, \bar{v}]\}$

Offline on-policy evaluation

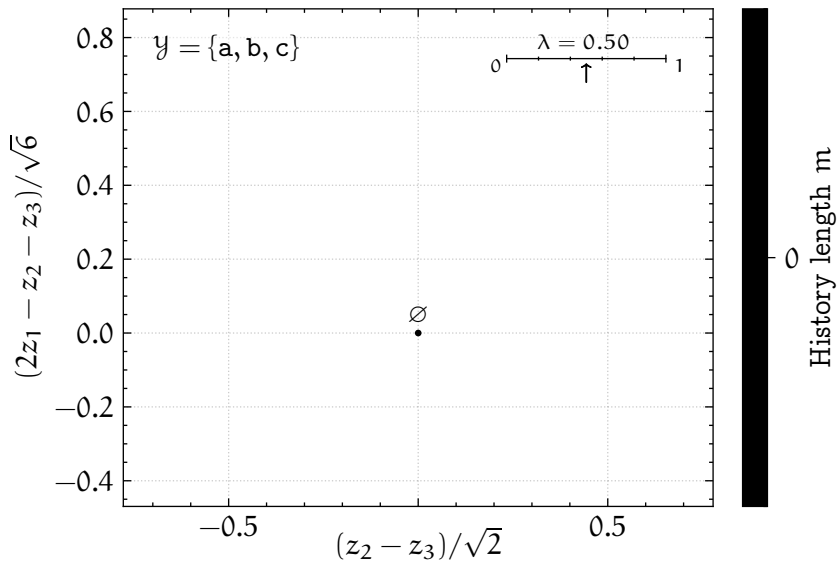
- ▶ We consider the problem of *policy evaluation* with offline data
 - The environment $\mathcal{E}$ is a hidden Markov model (no explicit policy)
 - The observation space $\mathcal{Y}$ is one-hot
- ▶ Q: How much data do we need to accurately estimate the value function?
- ▶ Given a function class $\mathcal{F} \subset \{\mathcal{Y}^\infty \rightarrow [\underline{v}, \bar{v}]\}$, find $f \in \mathcal{F}$ that minimizes

$$\mathcal{R}_{\mathcal{E}}(f) \doteq \mathbb{E}_{\mathcal{E}} \left[\left\{ f(y_0, y_{-1}, \dots) - \sum_{t=0}^{\infty} \gamma^t r(y_{t+1}) \right\}^2 \right],$$

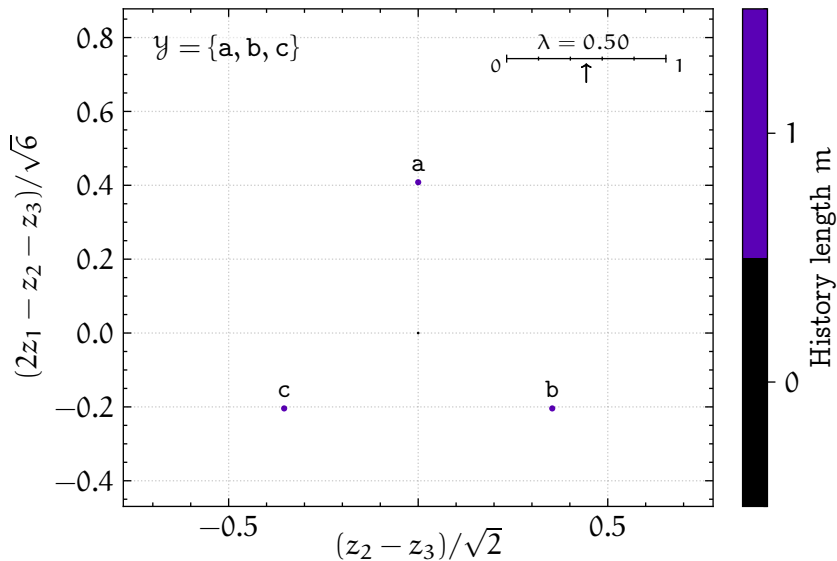
where $r : \mathcal{Y} \rightarrow [\underline{r}, \bar{r}]$ (rewards), $\gamma \in [0, 1)$ (discount), $\underline{v} \doteq \underline{r}/(1 - \gamma)$, and $\bar{v} \doteq \bar{r}/(1 - \gamma)$

- ▶ Window memory: $\mathcal{F}_m \doteq \{f \circ \text{win}_m \mid f : \mathcal{Y}^m \rightarrow [\underline{v}, \bar{v}]\}$
- ▶ Memory traces: $\mathcal{F}_\lambda \doteq \{f \circ z_\lambda \mid f : \mathcal{Z}_\lambda \rightarrow [\underline{v}, \bar{v}]\}$, where $\mathcal{Z}_\lambda \doteq \{z_\lambda(h) \mid h \in \mathcal{Y}^\infty\}$

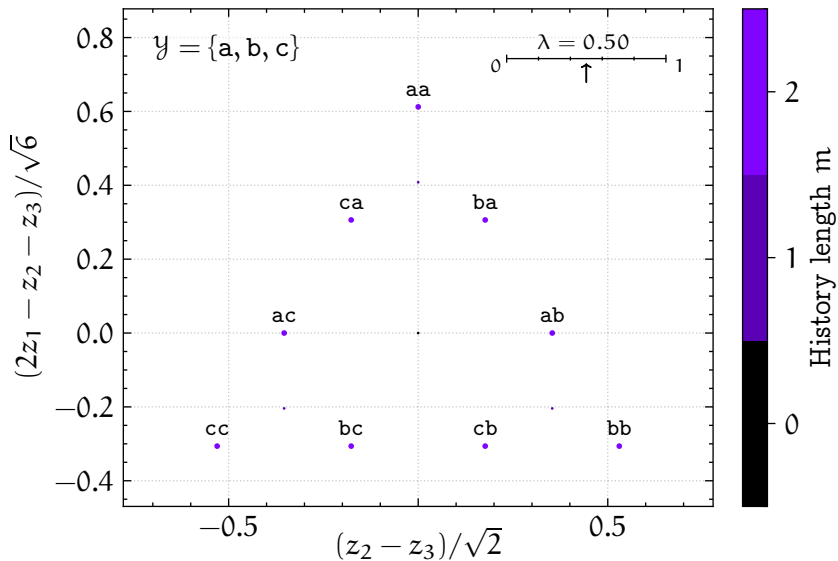
The geometry of the *trace space* $\mathcal{Z}_\lambda$



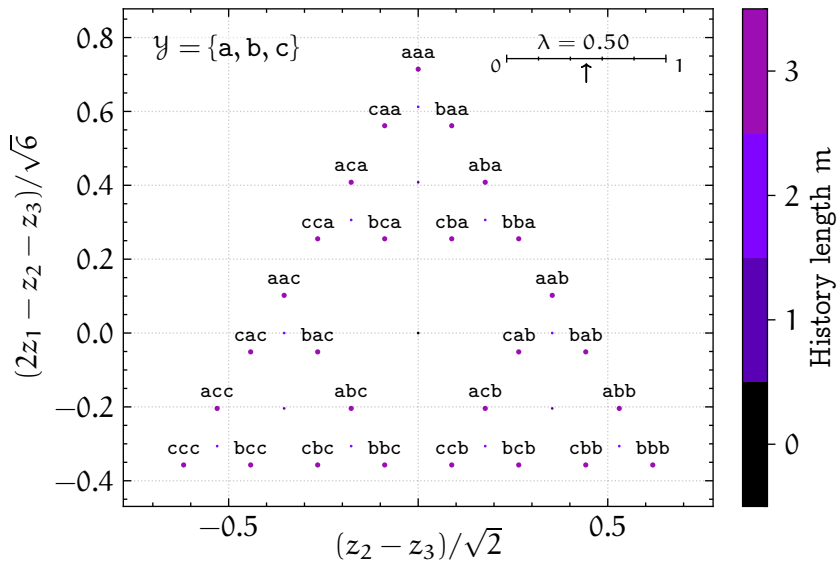
The geometry of the *trace space* $\mathcal{Z}_\lambda$



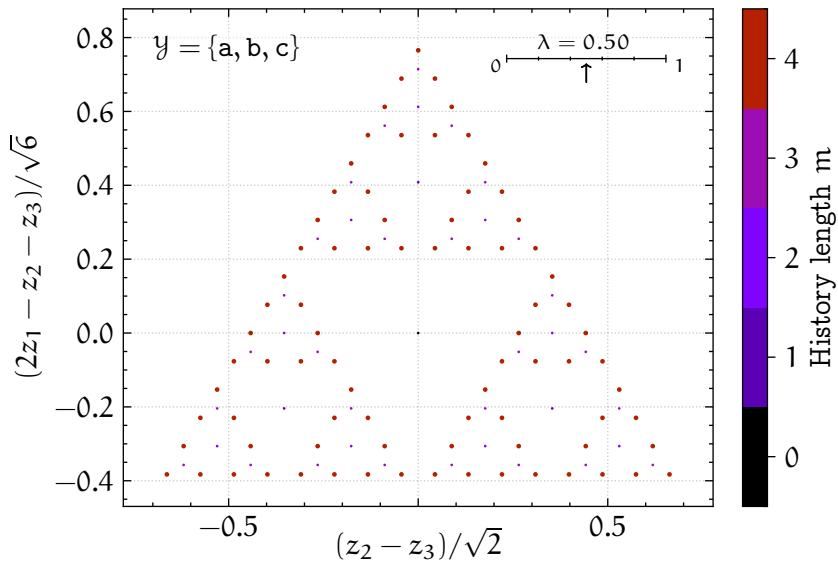
The geometry of the *trace space* $\mathcal{Z}_\lambda$



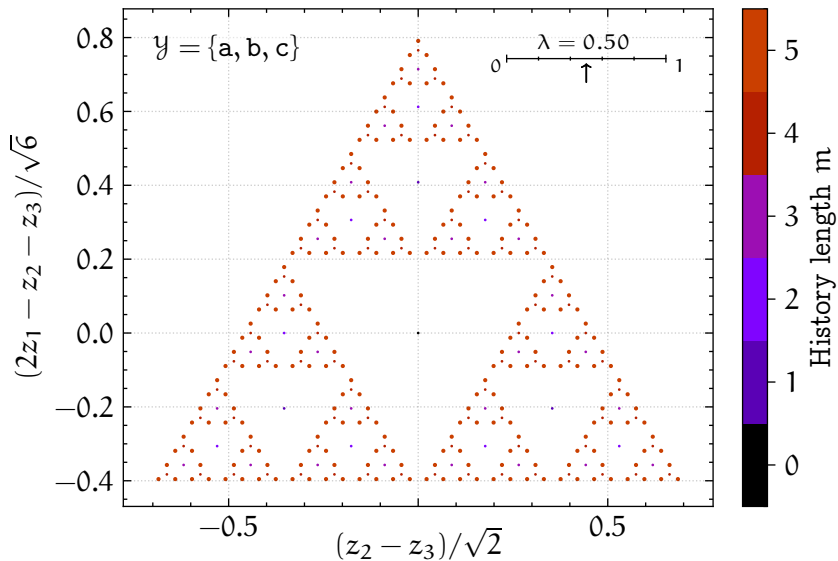
The geometry of the *trace space* $\mathcal{Z}_\lambda$



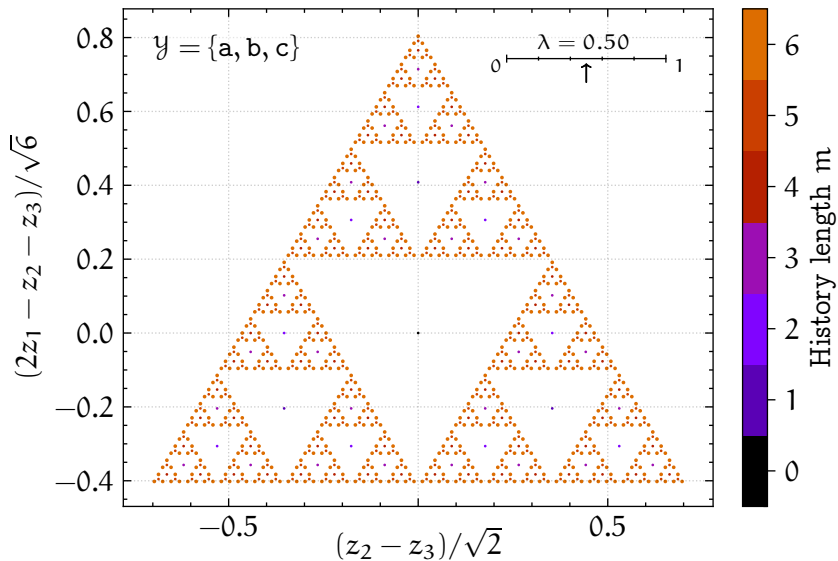
The geometry of the *trace space* $\mathcal{Z}_\lambda$



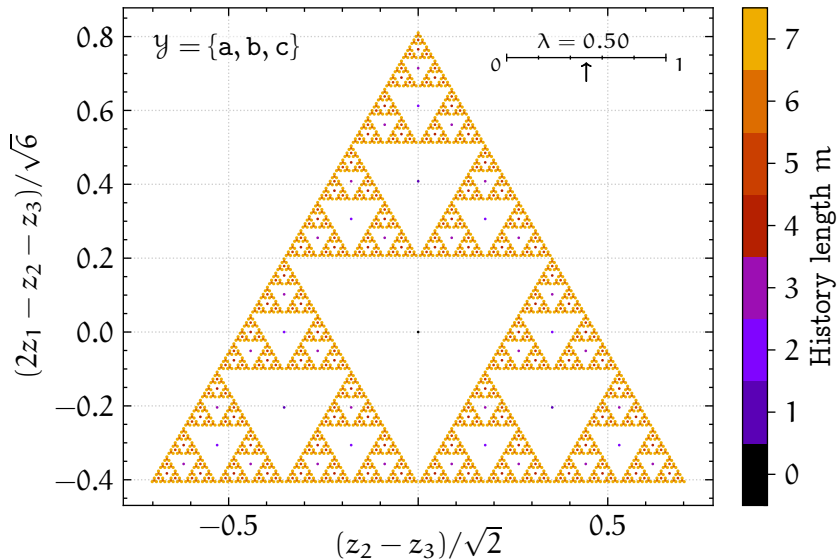
The geometry of the *trace space* $\mathcal{Z}_\lambda$



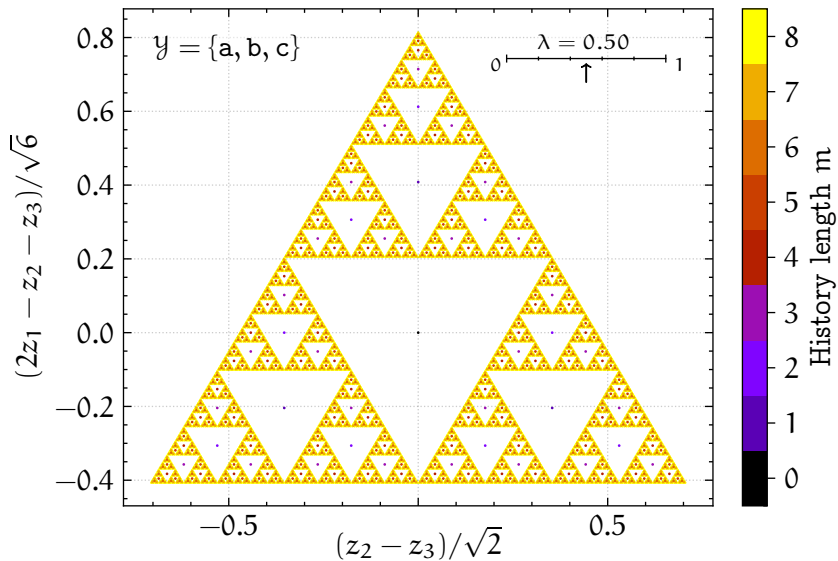
The geometry of the *trace space* $\mathcal{Z}_\lambda$



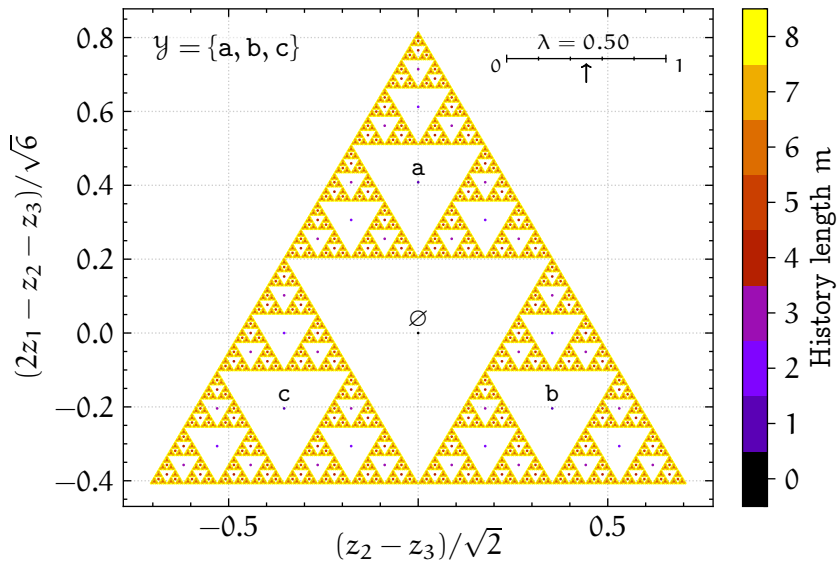
The geometry of the *trace space* $\mathcal{Z}_\lambda$



The geometry of the *trace space* $\mathcal{Z}_\lambda$



The geometry of the *trace space* $\mathcal{Z}_\lambda$



Learning value functions

Theorem (Hoeffding bound)

Given a dataset $\mathcal{D}$ of n trajectories from an environment $\mathcal{E}$, a function class $\mathcal{F}$, and some $\epsilon > 0$, let $\mathcal{F}^\epsilon$ be the smallest ϵ -cover of $\mathcal{F}$ and $f_n \doteq \arg \min_{f \in \mathcal{F}^\epsilon} \sum_{(x,y) \in \mathcal{D}} (f(x) - y)^2$. Then, with probability at least $1 - \delta$,
Learning a good value estimate seems easier if the *metric entropy* $H_\epsilon(\mathcal{F})$ of the function class $\mathcal{F}$ is small.

$$\mathcal{R}_\mathcal{E}(f_n) \leq \mathcal{R}_\mathcal{E}(\mathcal{F}) + (\bar{v} - \underline{v})^2 \sqrt{\frac{H_\epsilon(\mathcal{F}) + \log \frac{2}{\delta}}{2n}} + \mathcal{O}(\epsilon).$$

Let (X, ρ) be a metric space and $T \subset X$. The ϵ -covering number $N_\epsilon(T)$ is the cardinality of the smallest set $S \subset X$ such that for every $x \in T$ there exists a $y \in S$ with $\rho(x, y) \leq \epsilon$. The metric entropy of T is defined as $H_\epsilon(T) \doteq \log N_\epsilon(T)$.

Learning value functions

Theorem (Hoeffding bound)

Given a dataset $\mathcal{D}$ of n trajectories from an environment $\mathcal{E}$, a function class $\mathcal{F}$, and some $\epsilon > 0$, let $\mathcal{F}^\epsilon$ be the smallest ϵ -cover of $\mathcal{F}$ and $f_n \doteq \arg \min_{f \in \mathcal{F}^\epsilon} \sum_{\mathcal{D}} \{f(h) - \sum_{t=0}^{\infty} \gamma^t r(y_{t+1})\}^2$. Then, with probability at least $1 - \delta$,

$$\mathcal{R}_{\mathcal{E}}(f_n) \leq \mathcal{R}_{\mathcal{E}}(\mathcal{F}) + (\bar{v} - \underline{v})^2 \sqrt{\frac{H_{\epsilon}(\mathcal{F}) + \log \frac{2}{\delta}}{2n}} + \mathcal{O}(\epsilon).$$

Let (X, ρ) be a metric space and $T \subset X$. The ϵ -covering number $N_{\epsilon}(T)$ is the cardinality of the smallest set $S \subset X$ such that for every $x \in T$, there exists a $y \in S$ with $\rho(x, y) \leq \epsilon$. The metric entropy of T is defined as $H_{\epsilon}(T) \doteq \log N_{\epsilon}(T)$.

Metric entropy and memory

- For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!

Metric entropy and memory

- ▶ For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!
- ▶ Memory traces with forgetting factor $\lambda < \frac{1}{2}$ remember everything
 - There is no compression: z_λ is invertible and therefore $H_\epsilon(\mathcal{F}_\lambda) = \infty$

Metric entropy and memory

- ▶ For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!
- ▶ Memory traces with forgetting factor $\lambda < \frac{1}{2}$ remember everything
 - There is no compression: z_λ is invertible and therefore $H_\epsilon(\mathcal{F}_\lambda) = \infty$
- ▶ Need to “zoom in” to differentiate histories that only differ far in the past

Metric entropy and memory

- ▶ For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!
- ▶ Memory traces with forgetting factor $\lambda < \frac{1}{2}$ remember everything
 - There is no compression: z_λ is invertible and therefore $H_\epsilon(\mathcal{F}_\lambda) = \infty$
- ▶ Need to “zoom in” to differentiate histories that only differ far in the past
- ▶ The “resolution” of a function class is given by its Lipschitz constant

Metric entropy and memory

- ▶ For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!
- ▶ Memory traces with forgetting factor $\lambda < \frac{1}{2}$ remember everything
 - There is no compression: z_λ is invertible and therefore $H_\epsilon(\mathcal{F}_\lambda) = \infty$
- ▶ Need to “zoom in” to differentiate histories that only differ far in the past
- ▶ The “resolution” of a function class is given by its Lipschitz constant
- ▶ We consider the class $\mathcal{F}_{\lambda,L} \doteq \{f \circ z_\lambda \mid f : \mathcal{Z}_\lambda \rightarrow [\underline{v}, \bar{v}], f \text{ is } L\text{-Lipschitz}\}$

Metric entropy and memory

- ▶ For windows of length m , we have $H_\epsilon(\mathcal{F}_m) \in \Theta(|\mathcal{Y}|^m)$
 - Exponential in m : long windows are expensive!
- ▶ Memory traces with forgetting factor $\lambda < \frac{1}{2}$ remember everything
 - There is no compression: z_λ is invertible and therefore $H_\epsilon(\mathcal{F}_\lambda) = \infty$
- ▶ Need to “zoom in” to differentiate histories that only differ far in the past
- ▶ The “resolution” of a function class is given by its Lipschitz constant
- ▶ We consider the class $\mathcal{F}_{\lambda,L} \doteq \{f \circ z_\lambda \mid f : \mathcal{Z}_\lambda \rightarrow [\underline{v}, \bar{v}], f \text{ is } L\text{-Lipschitz}\}$

Lemma (metric entropy of $\mathcal{F}_{\lambda,L}$)

$$H_\epsilon(\mathcal{F}_{\lambda,L}) \in \mathcal{O}\left(L^{\min\{d_\lambda, |\mathcal{Y}|-1\}}\right) \quad \text{where} \quad d_\lambda \doteq \frac{\log |\mathcal{Y}|}{\log(1/\lambda)}$$

Fast forgetting: $\lambda < \frac{1}{2}$

Theorem (window $\rightarrow$ trace)

Let $m \in \mathbb{N}$ be a window length, $0 < \lambda < \frac{1}{2}$ a forgetting factor, and define

$$L(m) = \frac{\bar{v} - \underline{v}}{\sqrt{5}(1 - 2\lambda)\lambda^{m-1}}.$$

Windows are not more efficient than memory traces.

Then, for every $\epsilon > 0$ and every environment $\mathcal{E}$,

$$\mathcal{R}_{\mathcal{E}}(\mathcal{F}_{\lambda, L(m)}) \leq \mathcal{R}_{\mathcal{E}}(\mathcal{F}_m) \quad \text{and} \quad H_{\epsilon}(\mathcal{F}_{\lambda, L(m)}) \in \mathcal{O}(|\mathcal{Y}|^m) = \mathcal{O}(H_{\epsilon}(\mathcal{F}_m)).$$

Fast forgetting: $\lambda < \frac{1}{2}$

Theorem (window $\rightarrow$ trace)

Let $m \in \mathbb{N}$ be a window length, $0 < \lambda < \frac{1}{2}$ a forgetting factor, and define

$$L(m) = \frac{\bar{v} - \underline{v}}{\sqrt{2}(1 - 2\lambda)\lambda^{m-1}}.$$

Then, for every $\epsilon > 0$ and every environment $\mathcal{E}$,

$$\mathcal{R}_{\mathcal{E}}(\mathcal{F}_{\lambda, L(m)}) \leq \mathcal{R}_{\mathcal{E}}(\mathcal{F}_m) \quad \text{and} \quad H_{\epsilon}(\mathcal{F}_{\lambda, L(m)}) \in \mathcal{O}(|\mathcal{Y}|^m) = \mathcal{O}(H_{\epsilon}(\mathcal{F}_m)).$$

Fast forgetting: $\lambda < \frac{1}{2}$

Theorem (trace $\rightarrow$ window)

Let $\lambda \in [0, 1)$ be a forgetting factor, $L > 0$ a Lipschitz constant, $\epsilon \in (0, L)$, and define

$$m(\lambda, L) = \left\lceil \frac{\log(L/\epsilon)}{\log(1/\lambda)} \right\rceil.$$

Then, Memory traces with $\lambda < \frac{1}{2}$ seem no more efficient than windows.

$$\mathcal{R}_\epsilon(\mathcal{F}_{m(\lambda, L)}) \leq \mathcal{R}_\epsilon(\mathcal{F}_{\lambda, L}) + \mathcal{O}(\epsilon) \quad \text{and} \quad H_\epsilon(\mathcal{F}_{m(\lambda, L)}) \in \mathcal{O}(L^{d_\lambda}).$$

If $\lambda < \frac{1}{2}$, then $d_\lambda < |Y| - 1$.

Fast forgetting: $\lambda < \frac{1}{2}$

Theorem (trace $\rightarrow$ window)

Let $\lambda \in [0, 1)$ be a forgetting factor, $L > 0$ a Lipschitz constant, $\epsilon \in (0, L)$, and define

$$m(\lambda, L) = \left\lceil \frac{\log(L/\epsilon)}{\log(1/\lambda)} \right\rceil.$$

Then, for every environment $\mathcal{E}$,

$$\mathcal{R}_{\mathcal{E}}(\mathcal{F}_{m(\lambda, L)}) \leq \mathcal{R}_{\mathcal{E}}(\mathcal{F}_{\lambda, L}) + \mathcal{O}(\epsilon) \quad \text{and} \quad H_{\epsilon}(\mathcal{F}_{m(\lambda, L)}) \in \mathcal{O}(L^{d_{\lambda}}).$$

If $\lambda < \frac{1}{2}$, then $d_{\lambda} < |\mathcal{Y}| - 1$.

Slow forgetting: $\lambda \geq \frac{1}{2}$

Theorem (T-maze)

There exists a sequence $(\mathcal{E}_k)$ of environments (with constant observation space $\mathcal{Y}$) with the property that, for every $\epsilon > 0$,

Memory traces ($\lambda \geq \frac{1}{2}$) can be significantly more efficient than windows.

In particular, the *T-maze* with corridor length k is such a sequence. In this case, the minima are attained at $m_k = k$, $\lambda_k = \frac{k-1}{k}$, and $L_k \leq \sqrt{2ek}$.

Slow forgetting: $\lambda \geq \frac{1}{2}$

Theorem (T-maze)

There exists a sequence $(\mathcal{E}_k)$ of environments (with constant observation space $\mathcal{Y}$) with the property that, for every $\epsilon > 0$,

$$\min_{m \in \mathbb{N}} \{H_\epsilon(\mathcal{F}_m) \mid \mathcal{R}_{\mathcal{E}_k}(\mathcal{F}_m) = 0\} \in \Omega(|\mathcal{Y}|^k), \text{ and}$$
$$\min_{\lambda \in [0,1)} \min_{L \geq 0} \{H_\epsilon(\mathcal{F}_{\lambda,L}) \mid \mathcal{R}_{\mathcal{E}_k}(\mathcal{F}_{\lambda,L}) = 0\} \in \mathcal{O}(k^{|\mathcal{Y}|-1}).$$

In particular, the *T-maze* with corridor length k is such a sequence. In this case, the minima are attained at $m_k = k$, $\lambda_k = \frac{k-1}{k}$, and $L_k \leq \sqrt{2}ek$.

Forgetting: fast and slow

- For $\lambda < \frac{1}{2}$, learning with windows and traces seems equivalent

Forgetting: fast and slow

- ▶ For $\lambda < \frac{1}{2}$, learning with windows and traces seems equivalent
- ▶ For $\lambda \geq \frac{1}{2}$, there exist environments where traces are much more efficient

Why can traces be more efficient?

Forgetting: fast and slow

- ▶ For $\lambda < \frac{1}{2}$, learning with windows and traces seems equivalent
- ▶ For $\lambda \geq \frac{1}{2}$, there exist environments where traces are much more efficient

Why can traces be more efficient?

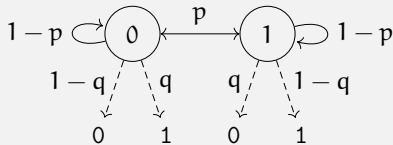
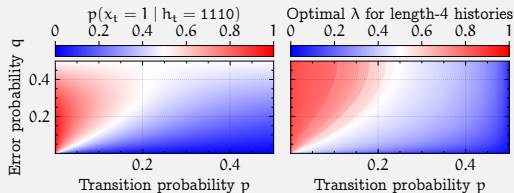
- ▶ In the T-maze, most of the $|y|^k$ histories are irrelevant $\rightarrow$ allows for small L

Forgetting: fast and slow

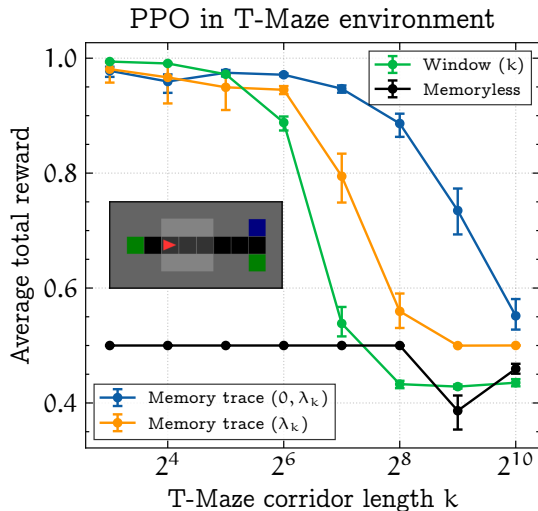
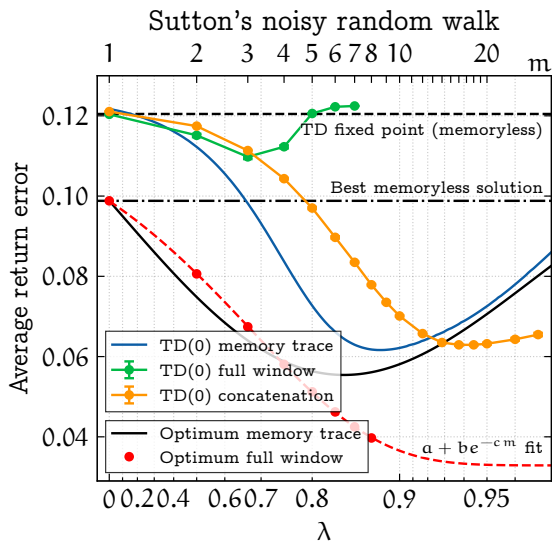
- ▶ For $\lambda < \frac{1}{2}$, learning with windows and traces seems equivalent
- ▶ For $\lambda \geq \frac{1}{2}$, there exist environments where traces are much more efficient

Why can traces be more efficient?

- ▶ In the T-maze, most of the $|y|^k$ histories are irrelevant $\rightarrow$ allows for small L
- ▶ In other environments, memory traces can effectively smooth out noise



Deep RL with memory traces



Summary



- ▶ Memory is important in partially observable environments
- ▶ We analyze *memory traces*: exponential moving averages of past observations
- ▶ There is a close connection to *sliding window* memory
- ▶ We prove that memory traces are strictly more powerful than sliding windows
- ▶ Memory traces are an effective drop-in replacement for *frame stacking*
- ▶ Paper, code & more at onnoeberhard.com/memory-traces

Thank you for listening!

Bibliography

-  Bakker, Bram (2001). “Reinforcement Learning with Long Short-Term Memory”.
In: *Advances in Neural Information Processing Systems*. Vol. 14. [LINK](#).